BITE403L Embedded Systems and IoT

Module 3

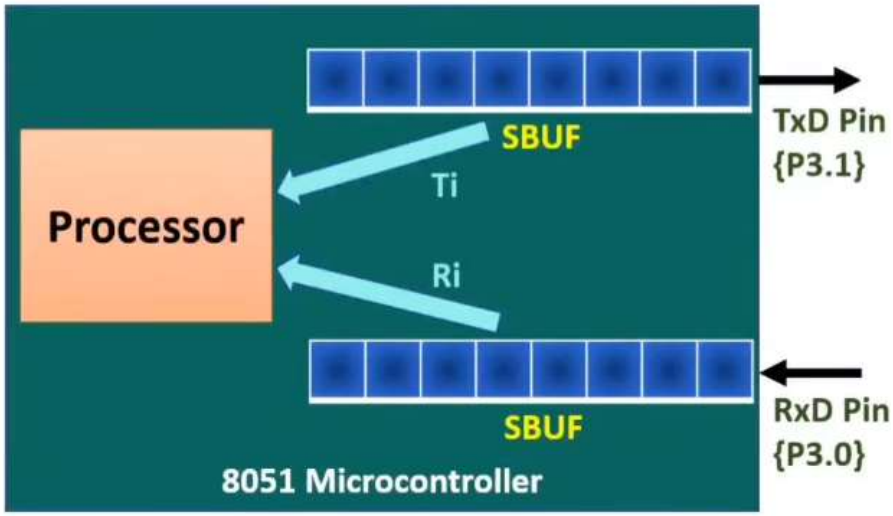
Dr.R.S.Rampriya,

SCORE, VIT.

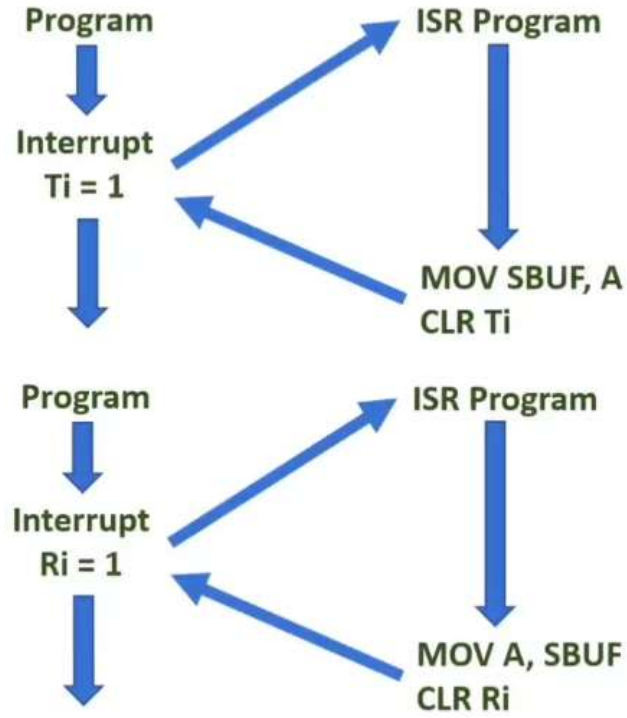
Port, Timer and Serial Programming

8051 Assembly Code

Serial Communication in 8051 MCU



- For Serial Communication, 8051 has two pins: TxD {P3.1} for serial transmission and RxD {P3.0} for serial reception.
- SBUF register {8 bits} will give and take data serially for serial communication on TxD and RxD.
- In serial communication, 1st it will send/receive LSB and at last it will send/receive MSB.
- Once, 1Byte transmission is completed, Ti interrupt tells processor 8 bits transmission is completed.
- Once, 1Byte Reception is completed, Ri interrupt tells processor 8 bits reception is completed.



- To configure serial communication, we need to configure SCON register of 8051.

SCON Register

✓ SCON Serial Control register –
{Bit Address SCON.7 to SCON.0}

SM0	SM1	SM2	REN	TB8	RB8	TI	RI

❑ SM0 & SM1 – Mode Control bits

SM0	SM1	Serial Mode	Description	Baud Rate
0	0	Mode 0	Shift Register	Fosc/12
0	1	Mode 1	8 bit UART	Variable
1	0	Mode 2	9 bit UART	Fosc/32 or Fosc/64
1	1	Mode 3	9 bit UART	Variable

❑ Mode 0 {Shift Register sends only data}

D0	D1	D2	D3	D4	D5	D6	D7

❑ Mode 1 {8 Bit UART, 1st Start bit 0, then 8bits data and at last stop bit 1}

0	D0	D1	D2	D3	D4	D5	D6	D7	1

❑ Mode 2 & 3 {9 Bit UART, 1st Start bit 0, then 8 bits data, 1 bit parity and at last stop bit 1}

0	D0	D1	D2	D3	D4	D5	D6	D7	P	1

❑ SM2 – Enables Multiprocessor System with Mode 2 and Mode 3.

❑ REN – Receiver Enable
❑ REN = 0, receiver disabled
❑ REN = 1, receiver enabled

❑ TB8 – Transmitted bit 8 {Technically it is programmable 9th bit in mode 2 and 3}
❑ Mode 0 – not used
❑ Mode 1 – stop bit '1'
❑ Mode 2 & 3 – Parity bit, programmed by programmer.

❑ RB8 – Received bit 8 {Technically it is programmable 9th bit in mode 2 and 3}
❑ Mode 0 – not used
❑ Mode 1 – stop bit '1'
❑ Mode 2 & 3 – Parity bit, programmed by programmer.

❑ RI – Receive Interrupt
❑ It will be one after SBUF receives 8 bits data.
❑ RI will be cleared by programmer in ISR program.

❑ TI – Transmit Interrupt
❑ It will be one after SBUF transmits 8 bits data.
❑ TI will be cleared by programmer in ISR program.

Code 1

Write an assembly language code for initializing P0 and P1 with 10H and 20H respectively. Then perform logical AND operation on these values. The result has to be displayed in P3.

```
MOV P0,#10H
MOV P1,#20H
MOV A,P0
ANL A,P1
MOV P3,A
```

When should I choose Timer 0 or Timer 1?

Requirement	Typical choice
Delay generation	Timer 0
General timing	Timer 0
Event counting	Timer 0 or Timer 1
UART baud-rate generation	Timer 1
Need both UART + another timer	Timer 1 for UART, Timer 0 for other timing

Important point

- There is **no rule saying Timer 0 must be used for delay**.
- You can use **either Timer 0 or Timer 1** for delay.
- The choice is often based on whether one timer is already occupied by another function.

When should I choose Mode 0, 1, 2, or 3?

- The timer modes are determined by **TMOD**.

Mode	Timer size	Main feature	Typical use
Mode 0	13-bit	13-bit timer	Rarely used
Mode 1	16-bit	Full 16-bit timer	Delays / long time intervals
Mode 2	8-bit	Auto-reload	Baud rate / repetitive delays
Mode 3	Split Timer 0	Two separate 8-bit timers	Special applications

UART baud-rate calculation

- For serial communication:
- Baud Rate= $\frac{\text{fosc}}{384(256-\text{T}\quad)}$
- This is specifically associated with the conventional Timer 1 Mode 2 baud-rate generation.
- Convert decimal 253 to hexadecimal:
- 253 = FDH

$$Baud\ Rate = \frac{f_{osc}}{384(256 - TH1)}$$

Substitute:

$$9600 = \frac{11.0592 \times 10^6}{384(256 - TH1)}$$

Therefore:

$$384(256 - TH1) = \frac{11.0592 \times 10^6}{9600}$$

$$384(256 - TH1) = 1152$$

Therefore:

$$256 - TH1 = \frac{1152}{384}$$

$$256 - TH1 = 3$$

Therefore:

$$TH1 = 256 - 3$$

$$\boxed{TH1 = 253}$$

Code 2

Write an Assembly language code for receiving data via serial ports and displaying in P3 using a 8051 microcontroller which functions with an oscillator frequency of 11.0592 MHz and at 9600 baud rate.

- The actual data flow is:
- **Serial Port → SBUF → Accumulator A → Port 3**
- So the program continuously performs:
- Configure Timer 1 for 9600 baud.
- Configure the serial port for 8-bit reception.
- Start Timer 1.
- Wait for a byte to arrive.
- Read the byte from SBUF.
- Send the byte to P3.
- Clear RI.
- Repeat continuously.

Cont…

Write an Assembly language code for receiving data via serial ports and displaying in P3 using a 8051 microcontroller which functions with an oscillator frequency of 11.0592 MHz and at 9600 baud rate.

MOV TMOD,#20H //0010 0000B- This configures Timer 1 in Mode 2, i.e., 8-bit auto-reload mode. Timer 1 is used to generate the baud rate.

MOV SCON,#50H //0101 0000B. **Serial Mode 1** → 8-bit UART, variable baud rate, REN = 1 → enables serial reception. So, the 8051 is now ready to receive serial data.

```
MOV TH1,#0FDH
SETB TR1
HERE: JNB RI, HERE
MOV A,SBUF
MOV P3,A
CLR RI
SJMP HERE
END
```

✓ TCON register – {Bit Address TCON.7 to TCON.0}

TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0

✓ TMOD register – Timer Mode Control register

GATE	C/$\overline{T}$	M1	M0	GATE	C/$\overline{T}$	M1	M0
Timer 1				Timer 0			

✓ SCON Serial Control register – {Bit Address SCON.7 to SCON.0}

SM0	SM1	SM2	REN	TB8	RB8	TI	RI

Code 3

Write an Assembly language code for transmitting a character "A" via serial ports using a 8051 microcontroller which functions with an oscillator frequency of 11.0592 MHz and at 2400 baud rate.

- We need to configure:
- Timer 1 → to generate the baud rate
- Timer 1 Mode 2 → 8-bit auto-reload
- Serial Mode 1 → 8-bit UART
- REN = 1 → receiver enabled
- TR1 = 1 → start Timer 1
- Put 'A' into SBUF
- Wait until transmission is complete using TI
- Repeat

Code 3

Write an Assembly language code for transmitting a character “A” via serial ports using a 8051 microcontroller which functions with an oscillator frequency of 11.0592 MHz and at 2400 baud rate.

```
MOV TMOD,#20H
MOV TH1,#F4H
MOV SCON,#50H
SETB TR1
AGAIN:MOV SBUF,#'A'
LOOP:JNB TI,LOOP
CLR TI
SJMP AGAIN
END
```

Code 4

Write an Assembly language code for transmitting a string "MESSAGE" continuously via serial ports using a 8051 microcontroller which functions with an oscillator frequency of 11.0592 MHz and at 4800 baud rate.

```
MOV TMOD,#20H
MOV TH1,#FAH
MOV SCON,#50H
SETB TR1
START: MOV A,#'M'
ACALL F1
MOV A,#'E'
ACALL F1
MOV A,#'S'
ACALL F1
MOV A,#'S'
ACALL F1
MOV A,#'A'
ACALL F1
MOV A,#'G'
ACALL F1
MOV A,#'E'
ACALL F1
SJMP START
F1: MOV SBUF,A
HERE:JNB TI,HERE
CLR TI
RET
END
```

Timer and Serial Programming

Embedded C

Program 1: Toggle to alternate bits of port 1

```
#include <REGX51.H>
#include <delay.h>
void main()
{
while(1)
{
P1=0x55;
DELAY_LOOP_Wait(period);
P1=0xAA;
DELAY_LOOP_Wait(period);
}
}
```

delay.h

```
//Delay file – save file as delay.h in project folder
void DELAY_LOOP_Wait(const unsigned int DELAY);
#define period 100
void DELAY_LOOP_Wait(const unsigned int DELAY)
{
unsigned int x, y;
for (x = 0; x <= DELAY; x++)
{
for (y = 0; y <= 120; y++);
}
}
```

```
/*-------------------------------------------------------*-
DELAY_LOOP_Wait()
Delay duration varies with parameter.
Parameter is, *ROUGHLY*, the delay, in milliseconds,
on 12MHz 8051 (12 osc cycles).
You need to adjust the timing for your application!
-*-------------------------------------------------------*/
```

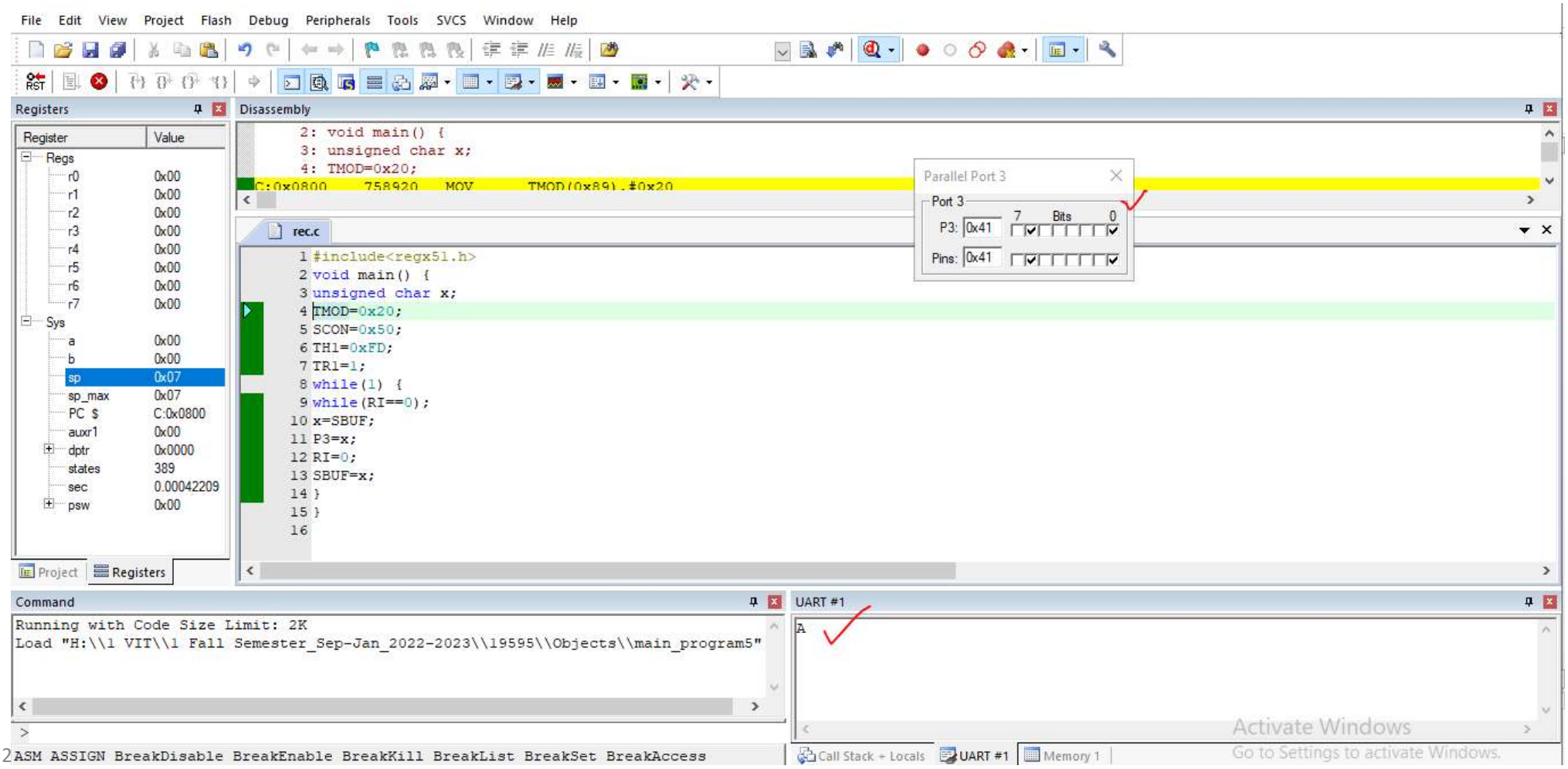
Program 2: Receiving an ASCII Character

Write an Embedded C for receiving data via serial ports and displaying in P3 using a 8051 microcontroller which functions with an oscillator frequency of 11.0592 MHz and at 9600 baud rate.

```
#include<regx51.h>
void main() {
unsigned char x;
TMOD=0x20;
SCON=0x50;
TH1=0xFD;
TR1=1;
while(1) {
while(RI==0);
x=SBUF;
P3=x;
RI=0;
SBUF=x;
}
}
```

Type Character ‘A’ in UART Window (view->serial window)

AH = 41H (of 65_{10} ASCII of A) = 01000001.



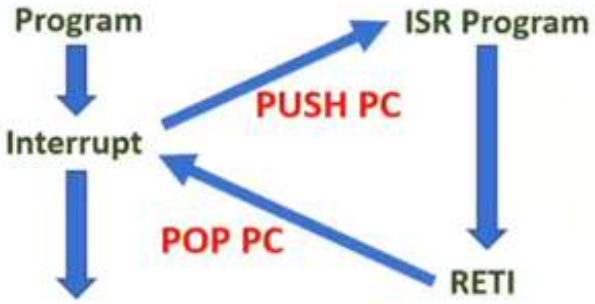
Interrupt Programming

ALP

Interrupts in 8051 MCU

❖ **Basics of Interrupts in 8051**

- ❑ 8051 has five interrupts and all are vectored interrupt.
- ❑ Two Hardware interrupts : $\overline{INT0}$ and $\overline{INT1}$
- ❑ Two Timer Overflow internal Interrupts : TF0 and TF1
- ❑ Serial Communication internal Interrupt : Common for RI and TI
- ❑ All the interrupts are controlled by IE and IP registers.



❖ **Priority and Vector Address of Interrupts in 8051**

Interrupt	Priority	Vector Address
$\overline{INT0}$	1	0003H
TF0	2	000BH
$\overline{INT1}$	3	0013H
TF1	4	001BH
Serial (RI or TI)	5	0023H

❖ **IE – Interrupt Enable Register {Bit Addressable IE.7 to IE.0}**

EA	-	ET2	ES	ET1	EX1	ET0	EX0

- ❑ EA – Enable All, ET2 – Reserved, ES – Enable Serial, ET1 – Enable Timer 1, EX1 – Enable $\overline{INT1}$, ET0 – Enable Timer 0 and EX0 – Enable $\overline{INT0}$.
- ❑ To Enable it, make it 1
- ❑ To Disable it, make it 0

❖ **IP – Interrupt Priority Register {Bit Addressable IP.7 to IP.0}**

-	-	PT2	PS	PT1	PX1	PT0	PX0

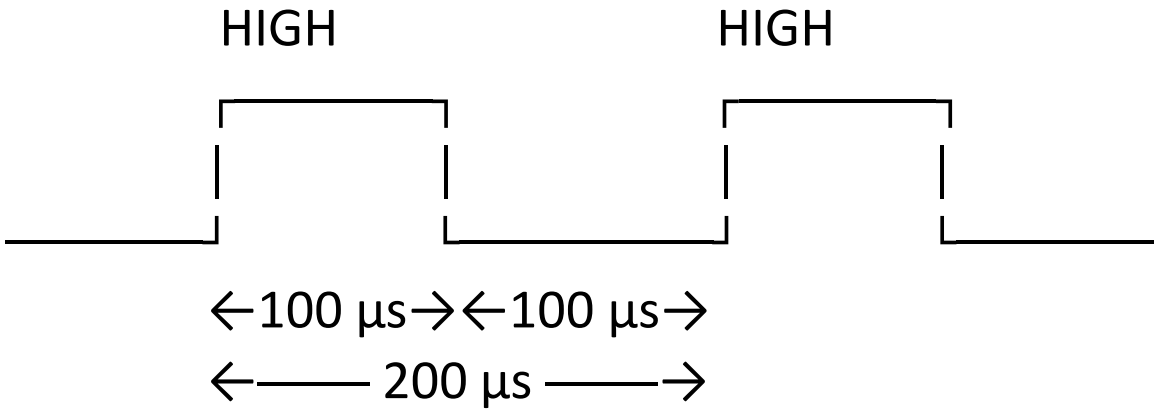
- ❑ PT2 – Reserved, PS – Priority Serial, PT1 – Priority Timer 1, PX1 – Priority $\overline{INT1}$, PT0 – Priority Timer 0 and PX0 – Priority $\overline{INT0}$.
- ❑ To have high Priority, make it 1
- ❑ To have low Priority, make it 0

Problem 1 – Timer Interrupt

- Write a program that continuously get 8-bit data from P1 and sends it to P2 while simultaneously creating a square wave of 200 µs period on pin P0.1. Use Timer 0 in mode 2 to create square wave. Assume that XTAL = 11.0592 MHz.
- **Given:**
- XTAL = 11.0592 MHz
- Timer 0 → Mode 2 (8-bit auto-reload)
- Required square-wave period = 200 µs
- Therefore, timer must overflow every **100 µs** because P0.1 toggles on each interrupt.

About Square Wave

- The question says: Generate a **square wave of 200 µs period** on P0.1.
- A square wave looks like this:



- One complete cycle consists of: **HIGH time + LOW time**
- For a square wave: $T_{HIGH} = T_{LOW}$
- Therefore: $T_{HIGH} = T_{LOW} = 200/2 = 100\mu s$

CPL

- Program uses: CPL P0.1; CPL means **complement/toggle**.
- So every time the timer overflows: Timer overflow → CPL P0.1 → Pin changes state
- Therefore:

  Overflow 1 → P0.1 = HIGH

  100 µs

  Overflow 2 → P0.1 = LOW

  100 µs

  Overflow 3 → P0.1 = HIGH

  100 µs

  ...
- Hence: Timer overflow interval=100µs and Square-wave period=200µs. So **200 µs is the required complete period**, not the required time between two toggles.

Recall 8051 Timer Modes

Mode	Timer size	Maximum count
Mode 0	13-bit	8192 (2^13)
Mode 1	16-bit	65536 (2^16)
Mode 2	8-bit auto-reload	256 (2^8)
Mode 3	Split timer	Depends

Count Calculation

- For 8051:
- Calculate Machine Cycle Time: $T_{machine}$=12/11.0592MHz=1.085 µs
- Required timer overflow interval: 100µs
- Calculate value of counts: counts (N) = delay/machine cycle time
- Number of timer counts: N = 100/1.085 = 92.16 ≈ **92 counts**
- For Mode 2: Reload value = 256−92 = 164 = A4H
- Therefore: **TH0 = A4H**

Timer 0 Mode 2

- We execute: MOV TH0, #0A4H
- Initially, the timer is loaded with: TH0 = A4H; TL0 = A4H
- Then: SETB TR0 //starts Timer 0.
- The timer counts: A4 → A5 → A6 → A7 → ... → FE → FF → overflow
- That's approximately **92 counts**.
- At overflow: TL0 → automatically reloads from TH0 So: TL0 = A4H again.
- This is why Mode 2 is called: **8-bit auto-reload mode**
- We don't need to manually reload the timer every time.

Enable Timer 0 interrupt

- MOV IE, #82H
- Convert 82H to binary: 82H = 1000 0010B
- 8051 IE register:

```
EA   -   ET2 ES ET1 EX1 ET0 EX0
|                           |
1                           1
```

- So:
- EA = 1 → Enable global interrupts
- ET0 = 1 → Enable Timer 0 interrupt
- Therefore Timer 0 interrupts are enabled.

Interrupt Vector Address

- The 8051 reserves specific locations for interrupts.
- For example:

Event	Vector address
Reset	0000H
External Interrupt 0	0003H
Timer 0	000BH
External Interrupt 1	0013H
Timer 1	001BH
Serial	0023H

- Therefore, ORG 000BH is used because we are using **Timer 0 interrupt**.

Problem Logic

Main Task

Main Program continuously transfers:

P1 → Accumulator → P2

At the same time, Timer 0 is running independently.

When Timer 0 overflows:

Timer 0 overflow

↓

Interrupt

↓

ISR

CPL P0.1

↓

RETI

↓

Return to main program

Main task: P1 → P2

Interrupt task: Timer 0 → P0.1 toggle

8051 ALP

```
ORG 0000H
      LJMP MAIN
 ORG 000BH
      CPL P0.1
      RETI
 ORG 0030H
MAIN:   MOV TMOD, #02H
      MOV P1, #0FFH
      MOV P2, #00H
      MOV TH0, #0A4H
      MOV IE, #82H
      SETB TR0
 BACK:   MOV A, P1
      MOV P2, A
      SJMP BACK
      END
```

Problem 2 – External Hardware Interrupt

- Write a program that continuously get 8-bit data from P1 and sends it to P2 while the INT1 pin is connected to a switch which is normally high. Whenever it goes LOW, it should turn ON a LED. The LED is connected to P0.3 and is normally OFF. When it is turned ON, it should stay ON for fraction of second. As long as the switch is held low, the LED should stay ON.

Given Data

- Continuously read 8-bit data from Port P1 & send it to P2 (P1 -> P2)
- Use INT1
- Switch -> Normally High
- Goes Low when pressed -> LED ON
  - INT1 -> H -> L
  - IE -> 10000100 -> 84H
- LED is connected to P0.3
  - Normally OFF
  - Turns ON when interrupt occurs
  - Stays ON while switch is LOW / Stays ON for some time

About External Hardware Interrupts

2 pins in 8051:

Pin	Interrupt	Vector
Pin 12 (P3.2)	INT0	0003H
Pin 13 (P3.3)	INT1	0013H

2 activation levels:

- INT0/INT1 in level-triggered mode: Interrupt occurs when the pin is LOW.
- INT0/INT1 in edge-triggered mode: Interrupt occurs on a HIGH → LOW transition.

8051 ALP

```
ORG 0000H
      LJMP MAIN
ORG 0013H
      SETB P0.3       ; (turn on LED)
      MOV R3, #FFH
BACK:  DJNZ R3, BACK    ; (Create Delay)
          CLR P0.3
          RETI
ORG 0030H
MAIN:   MOV P1, #0FFH
          MOV P2, #00H
          CLR P0.3       ; (LED OFF)
           MOV IE, #84H   ;(Interrupt Occurs)
HERE:   MOV A, P1
          MOV P2, A
          SJMP HERE
```

Problem 3 – Serial Interrupt

- **Vector Address:** 0023H
- Write a program in which the 8051 reads data from P1 and writes it to P2 continuously while giving a copy of it to the serial COM port to be transferred serially. Assume that XTAL = 11.0592 MHz. Set the baud rate at 9600.

8051 ALP

```
ORG 0000H
      LJMP MAIN
ORG 0023H
      LJMP SERIAL
ORG 0030H
MAIN:   MOV P1, #0FFH
      MOV P2, #00H
      MOV TMOD, #20H       ; TIMER 1, MODE 2
      MOV TH1, #0FDH
      MOV SCON, #50H
     MOV IE, #90H
            SETB TR1
HERE:     MOV A, P1
          MOV SBUF, A
          MOV P2, A
          SJMP HERE
          END.
SERIAL:   JB TI, TRANS
          MOV P3, SBUF
          CLR RI
          RETI
TRANS:    CLR TI
          RETI
          END
```

Practise Questions on Serial Programming

Question 1

Write an 8051 ALP to receive serial data and display it on Port 3. Use Timer 1 in Mode 2 for baud-rate generation. Calculate TH1 for 2400 baud with fosc = 11.0592 MHz.

Question 2

A security system uses an 8051 microcontroller to transmit the message “SECURE” continuously through its serial port. Write an Assembly Language Program (ALP) to transmit the message at a 2400 baud rate using an oscillator frequency of 11.0592 MHz. Use a subroutine with the ACALL instruction for serial transmission.

Question 3

A water tank monitoring system uses an 8051 microcontroller with a 12 MHz crystal to receive water-level readings from multiple sensor units through a common serial bus at 9600 bps. Each frame contains an 8-bit water-level value and a 9th bit for identifying data and address/control frames. Develop an 8051 assembly language program to process valid data frames and display the water-level value on Port P1. If the received water-level value is below 60H, turn ON a pump indicator LED connected to P3.2; otherwise, turn it OFF. Address/control frames should not affect the display or pump control.

Question 4

Write an Embedded C program to find the largest number among 6 numbers stored starting at RAM location 51H and output the result to Port 0.

Question 5

Write an Embedded C program to blink an LED on Port 1 every 2 ms using Timer 0, Mode 0, with fosc = 11.0592 MHz.

Question 6

Write an Embedded C program to generate a 20 µs delay, then set logic 1 on bit 0 of Port 2. Use fosc = 24 MHz.

Question 7

Assume Port0 is the input port and an input device is connected to it. Write Embedded C program to read data from the input device via Port 0 and test whether it is greater than or lesser than a threshold value of 35. According to the result, place the input value into R3 or R4 respectively after a time delay of 2 ms.

Question 8

An embedded system has two status LEDs:

- Green—P1.0—2ms blink period
- Red — P1.2 — 40 ms blink period
- Both LEDs must blink independently.

Select the most appropriate timer configuration, justify it, calculate the preload values, and write the 8051 program. fosc = 11.0592 MHz.

Practise Questions on Interrupt Programming

Question 1

An 8051-based industrial machine controller uses an 11.0592 MHz crystal. An emergency-stop switch is connected to INT0 (P3.2), and an indicator LED is connected to P1.0. Write an 8051 assembly language program using an external hardware interrupt INT0 so that when the switch is pressed, the LED is turned ON for a short delay and then turned OFF. Configure INT0 as an edge-triggered interrupt, enable the required interrupts, and write the appropriate ISR.

Question 2

An 8051-based motor controller uses a 14.7456 MHz crystal and communicates with a control terminal at 9600 bps. Write an 8051 assembly language program using the serial interrupt to receive commands continuously. If the received character is 'S', turn ON a motor connected to P1.5; if the received character is 'O', turn it OFF. Configure the UART, generate the required baud rate using Timer 1, enable the serial interrupt, and process the received commands in the ISR.

Question 3

An 8051-based warning system uses a 12 MHz crystal, with a buzzer connected to P2.3. Write an 8051 assembly language program using Timer 1 in Mode 1 and a timer interrupt to toggle the buzzer every 20 ms. Configure Timer 1, calculate the initial values of TH1 and TL1, enable the Timer 1 and global interrupts, reload the timer inside the ISR, and toggle P2.3 after the required number of interrupts. Show the calculations clearly.

THANK YOU